

Ex1a: Text Generation using N-gram Language model

Learning Objective:

Implement an N-Gram-based predictive text model using bigram and trigram probability distributions. This helps to understand how contextual probability influences next-word prediction.

Steps:

1. Initialize the environment by installing and importing the Natural Language Toolkit (`nltk`). Download the **Brown Corpus** (for text data) and the **Universal Tagset** (for simplified Part-of-Speech tags).
2. Extract raw sentences from the Brown Corpus using `brown.sents()`.
3. **Tokenization:** Convert all words to lowercase and flatten the nested list into a single list of `tokens` to verify the total volume of training data.
4. Build an N-Gram Model for Bigram and Trigram.
5. Predict Next Word and display Top-5 Suggestions
6. Test Bigram & Trigram Prediction on certain words.

Program:

```
import nltk
import random
from nltk.corpus import brown
from nltk.util import ngrams
from collections import defaultdict, Counter

nltk.download('brown')

sentences = brown.sents()
sentences = [[word.lower() for word in sent] for sent in sentences]

def build_ngram_model(sentences, n=3):
    model = defaultdict(Counter)
    for sentence in sentences:
        sentence = ['<s>'] * (n - 1) + sentence + ['</s>']
        for gram in ngrams(sentence, n):
            context = gram[:-1]
            target = gram[-1]
            model[context][target] += 1
    return model

ngram_model = build_ngram_model(sentences, n=3)

def predict_next_word_probabilities(context, model):
    context = tuple(context)
    if context not in model:
        return {}
```

```

    total = sum(model[context].values())
    return {word: count / total for word, count in model[context].items()}

def sample_next_word(probabilities):
    words = list(probabilities.keys())
    probs = list(probabilities.values())
    return random.choices(words, probs)[0]

def generate_sentence(start_words, model, max_length=15):
    sentence = start_words[:]
    for _ in range(max_length):
        context = tuple(sentence[-2:])
        probs = predict_next_word_probabilities(context, model)
        if not probs:
            break
        next_word = sample_next_word(probs)
        if next_word == '</s>':
            break
        sentence.append(next_word)
    return " ".join(sentence)

context = ['i', 'am']
probs = predict_next_word_probabilities(context, ngram_model)

for word, prob in sorted(probs.items(), key=lambda x: -x[1]):10]:
    print(f"{word} : {prob:.4f}")

print(generate_sentence(['i', 'am'], ngram_model))
print(generate_sentence(['the', 'government'], ngram_model))
print(generate_sentence(['this', 'is'], ngram_model))

```

Output:

```

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
Next word probabilities for context: ['i', 'am']
not : 0.1159
a : 0.0628
sure : 0.0483
. : 0.0242
an : 0.0242
innocent : 0.0242
told : 0.0193
very : 0.0193
getting : 0.0193
' ' : 0.0193

```

Generated Sentences:

i am here concerned more with policy than with chaucer and cimabue , or
nearly .
the government unless made after a while with his distinguished colleague ,
mitropoulos , he limped .
this is best

Learning Outcome:

Upon completion of this experiment, the bigram and trigram probability distributions for next-word prediction was implemented and evaluated using the Brown Corpus.

Ex1b: Hidden Markov Model (HMM) based Predictive Text System

Learning Objective:

To implement a predictive text system using Hidden Markov Model (HMM), enabling students to understand contextual probability, POS transitions, and next-word prediction using the Brown Corpus.

Steps:

1. Initialize the environment by installing and importing the Natural Language Toolkit (nltk). Download the Brown Corpus (for text data) and the Universal Tagset (for simplified Part-of-Speech tags).
2. Extract raw sentences from the Brown Corpus using `brown.sents()`.
3. Tokenization: Convert all words to lowercase and flatten the nested list into a single list of tokens to verify the total volume of training data.
4. Load the tagged version of the corpus (`brown.tagged_sents`) which provides the "Hidden States" (POS tags) linked to the "Observations" (words).
5. Train the HMM Model. Use the `HiddenMarkovModelTrainer` to perform Supervised Learning. Calculate the internal parameters of the HMM: Initial State Probabilities, Transition Probabilities, Emission Probabilities

6. Build POS Transition Probabilities
7. Build Emission Probabilities
8. Define Prediction Function
9. Test the Model
10. Verify POS prediction on certain words.

Program:

```
import nltk
from nltk.corpus import brown
from collections import defaultdict, Counter

nltk.download('brown')
nltk.download('universal_tagset')

tagged_sentences = brown.tagged_sents(tagset='universal')[:3000]

transition_counts = defaultdict(Counter)
emission_counts = defaultdict(Counter)

for sent in tagged_sentences:
    prev_tag = "<START>"
    for word, tag in sent:
        word = word.lower()
        transition_counts[prev_tag][tag] += 1
        emission_counts[tag][word] += 1
        prev_tag = tag
    transition_counts[prev_tag]["<END>"] += 1

transition_prob = {}
for prev_tag, counter in transition_counts.items():
    total = sum(counter.values())
    transition_prob[prev_tag] = {tag: c / total for tag, c in counter.items()}

emission_prob = {}
for tag, counter in emission_counts.items():
    total = sum(counter.values())
    emission_prob[tag] = {word: c / total for word, c in counter.items()}

def guess_pos(word):
    word = word.lower()
    best_tag, best_prob = None, 0
    for tag, words in emission_prob.items():
        if word in words and words[word] > best_prob:
            best_tag, best_prob = tag, words[word]
    return best_tag

def predict_next_word_probabilities(context_sentence, top_k=15):
    last_word = context_sentence.strip().split()[-1].lower()
    prev_tag = guess_pos(last_word)
```

```

if prev_tag is None:
    return []
results = []
for next_tag, tag_prob in transition_prob[prev_tag].items():
    if next_tag == "<END>":
        continue
    for word, emit_prob in emission_prob[next_tag].items():
        results.append((word, next_tag, tag_prob * emit_prob))
total_prob = sum(p for _, _, p in results)
results = [(w, t, p / total_prob) for w, t, p in results]
results.sort(key=lambda x: -x[2])
return results[:top_k]

```

```

context = "The government is a great"
predictions = predict_next_word_probabilities(context)

```

```

for word, tag, prob in predictions:
    print(f"{word:12s} | {tag:6s} -> {prob:.4f}")

```

Output:

```

[nltk_data] Downloading package brown to /root/nltk_data...
[nltk_data]   Package brown is already up-to-date!
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Package universal_tagset is already up-to-date!

```

Next word probabilities for: 'The government is a great'

```

,          | .      -> 0.0215
and        | CONJ   -> 0.0187
of         | ADP    -> 0.0169
.          | .      -> 0.0168
in         | ADP    -> 0.0115
to         | PRT    -> 0.0097
mrs.       | NOUN   -> 0.0080
for        | ADP    -> 0.0061
to         | ADP    -> 0.0050
mr.        | NOUN   -> 0.0047
state      | NOUN   -> 0.0043
year       | NOUN   -> 0.0041
on         | ADP    -> 0.0038
at         | ADP    -> 0.0036
home       | NOUN   -> 0.0032

```

Learning Outcome:

Upon completion of this experiment, HMM based model was built to predict the next word with POS-based suggestions, and evaluated the effectiveness of context-driven predictive text generation using the Brown Corpus.